KnowledgeBase AI — Implementation Manual

6-Phase Architecture Blueprint, Engineering Rationale & Verification

Version 1.2 • Go + Fiber v2 + PostgreSQL + pgvector + Redis + Asynq + Gemini API

1. Executive Summary & Purpose

KnowledgeBase AI is an enterprise-grade, multi-tenant Retrieval-Augmented Generation (RAG) backend. Unlike trivial AI wrappers that simply forward uploaded documents directly into prompts, KnowledgeBase AI is engineered for production robustness, strict tenant isolation, asynchronous durable ingestion, recoverable background pipelines, and verifiable citation lineage.

This specification organizes the complete lifecycle of the platform into 6 sequential phases punctuated by 5 concrete checkpoints. Each phase addresses specific engineering failure modes, enforces strict interface boundaries, and guarantees database invariants before proceeding.

> **Core Architectural Principles**
> 1. Asynchronous Ingestion: The HTTP API never performs embedding generation or document parsing inline.
> 2. Durable Outbox Pattern: Ingestion jobs and outbox events are committed transactionally with document metadata.
> 3. Strict Multi-Tenancy: Every single database query and vector search is scoped to the authenticated organization.
> 4. Untrusted Model Output: LLM responses cannot bypass authorization or invent fabricated citation IDs.
> 5. Process Isolation: Fiber API and Asynq background workers share the codebase but run as isolated services.

2. Technology Stack & Component Matrix

Layer	Technology	Design Purpose / Rationale
Language	Go 1.22+	Memory efficiency, compile-time type safety, exceptional concurrency.
HTTP Framework	Fiber v2 (fasthttp)	High-performance routing; safe context isolation for background jobs.
Primary Database	PostgreSQL 16	ACID transactional state, outbox events, and user/org ownership.
Vector Search	pgvector (HNSW)	Cosine similarity search over 768-dimensional Gemini embeddings.
SQL Access	pgx/v5 (pgxpool)	Direct parameterized queries without ORM latency overhead.
Task Queue	Redis + Asynq	Durable task retry policies, uniqueness deduplication, priority queues.
Authentication	JWT + Refresh Sessions	15-minute access JWTs; rotating hashed refresh tokens with family revocation.
AI Provider	Gemini API (1.5 Pro / Embeddings)	Encapsulated provider interface requesting 768-dim embeddings.

3. The 6-Phase Breakdown & 5 Checkpoints

To deliver a reliable backend without accumulating technical debt, implementation proceeds strictly across 6 sequential phases. Transitions between phases are guarded by 5 explicit verification checkpoints.

Phase	Scope & Description	Checkpoint Gate	Status
Phase 1	Repository Foundation & Infrastructure	Checkpoint 1: Compiling binaries, Fiber router, migrations, health probes	COMPLETED
Phase 2	Identity, Authentication & Tenancy	Checkpoint 2: Argon2id, JWT/refresh rotation, org RBAC, cross-tenant tests	COMPLETED
Phase 3	Document Uploads & Outbox Pipeline	Checkpoint 3: Streaming upload, SHA-256, atomic storage/DB persistence	COMPLETED
Phase 4	Asynq Worker & Vector Ingestion	Checkpoint 4: Dispatcher, PDF/TXT/MD parsers, chunking, pgvector writes	PLANNED
Phase 5	Vector Search & RAG Chat Engine	Checkpoint 5: Cosine search, Gemini generation, verified citations, SSE streaming	PLANNED
Phase 6	Hardening, Observability & Evaluation	Final Release: Rate limits, Prometheus metrics, E2E tests, AI benchmark suite	PLANNED

Detailed Phase Analysis: What, How, and For What Purpose

Phase 1: Repository Foundation & Core Infrastructure (Checkpoint 1) — COMPLETED

- **What:** Go module structure, Fiber v2 router, configuration loader, PostgreSQL & Redis connection pools, database migrations, and health check endpoints.
- **How:** pgxpool with connection limits, go-redis client, standard error envelopes, and a multi-stage Dockerfile running non-root.
- **Purpose:** Provides a rock-solid, production-ready substrate ensuring external services can be probed and scaled independently.

Phase 2: Identity, Multi-Tenancy & Authentication (Checkpoint 2) — COMPLETED

- **What:** User registration, Argon2id password hashing, login, rotating opaque refresh tokens, JWT access tokens, organization lifecycle, and RBAC memberships (owner, admin, member).
- **How:** Argon2id (64MB memory, 1 iteration, 4 parallelism); 15m JWT access tokens; cryptographically random refresh tokens stored as SHA-256 hashes in PostgreSQL; family_id reuse revocation; single-transaction organization and owner creation; route-level RequireOrgMembership middleware.
- **Purpose:** Enforces absolute tenant isolation, prevents unauthorized cross-organization data access, and eliminates session hijacking risks.

Phase 3: Document Management & Ingestion Outbox (Checkpoint 3) — COMPLETED

- **What:** Streaming multipart uploads, file validation (PDF, TXT, MD), SHA-256 deduplication, atomic file storage, single-transaction document/job/outbox persistence, 202 Accepted ingestion entry point, document status, list, deletion, and retry.
- **How:** Streams raw bytes directly into storage via io.Pipe with simultaneous SHA-256 computation; inspects first 512 bytes for file magic signatures (%PDF-); executes an atomic PostgreSQL transaction inserting document, ingestion_job, and outbox_event; implements orphan file cleanup upon transaction rollback; supports Idempotency-Key headers.
- **Purpose:** Eliminates memory exhaustion DOS vulnerabilities on large uploads, prevents race conditions between storage and database state, and guarantees reliable ingestion handoff.

Phase 4: Asynq Ingestion Worker & Vector Ingestion (Checkpoint 4)

- **What:** Background task worker consuming outbox events, document parsing, text chunking, and vector embedding generation.
- **How:** Polls unpublished outbox events using 'FOR UPDATE SKIP LOCKED', enqueues deterministic tasks into Redis Asynq, calls Gemini embeddings (768 dimensions), and writes chunks to pgvector with HNSW indexing.
- **Purpose:** Decouples heavy file parsing and AI API latencies from HTTP response times while guaranteeing at-least-once processing.

Phase 5: Retrieval-Augmented Generation & Chat Engine (Checkpoint 5)

- **What:** Semantic vector search, context prompt engineering, Gemini chat generation, and Server-Sent Events (SSE) streaming.
- **How:** Computes query embeddings, queries pgvector strictly scoped by organization_id with cosine similarity and relevance thresholds, injects retrieved passages into guarded prompts, and validates citation chunk IDs against the database.
- **Purpose:** Eliminates LLM hallucinations by enforcing citations linked strictly to retrieved, authenticated passages.

Phase 6: Production Hardening, Observability & Evaluation (Final Release)

- **What:** Atomic Redis rate limiting, Prometheus metrics, structured request tracing, comprehensive integration tests, and RAG evaluation benchmarks.
- **How:** Redis sliding-window rate limiters per org/user/endpoint, automated test suites, and RAG benchmark scoring for faithfulness, recall, and latency.
- **Purpose:** Guarantees production stability under load and ensures predictable LLM cost and accuracy bounds.

4. Phase 3 Implementation & Engineering Architecture

Phase 3 establishes the ingestion ingestion entry point and document lifecycle engine. The implementation guarantees that uploads are accepted durably without blocking HTTP threads on heavy AI computations.

- **Streaming Upload & SHA-256 Pipe:** Uploads are streamed directly to storage via io.Pipe with real-time SHA-256 calculation. The server never buffers entire files into RAM, strictly enforcing the MAX_UPLOAD_BYTES limit.
- **Magic Byte & MIME Validation:** Validates actual file contents by inspecting magic bytes: PDFs must begin with '%PDF-', while Plain Text and Markdown must be valid UTF-8. Spoofed file extensions (.exe renamed to .pdf) are rejected with 400 INVALID_FILE_TYPE.
- **Transactional Outbox Pattern:** Document metadata, initial ingestion job, and outbox event are inserted in a single PostgreSQL transaction. The API returns 202 Accepted only after the transaction successfully commits.
- **Orphan File Prevention:** If the database transaction encounters an error (e.g., checksum conflict), the stored file is immediately deleted from the storage volume, ensuring zero orphaned files.
- **Client-Safe Status Masking:** GET /api/v1/documents/{documentID} provides sanitized document metadata. Internal filesystem paths, storage keys, and database connection strings are never leaked.
- **Idempotent Uploads:** Clients providing an Idempotency-Key header receive deterministic, cached responses without creating duplicate records or competing ingestion jobs.
- **Document Deletion & Retry Lifecycle:** DELETE /documents/{id} immediately flags status as 'deleting' (excluding it from retrieval) and queues a deletion event. POST /documents/{id}/retry safely resets failed ingestion jobs.

5. Checkpoint 3 Verification Report

Checkpoint 3 Verification Results: ALL TESTS PASSED

 Streaming Upload: Successfully accepted valid PDF, streamed to storage, and returned 202 Accepted.
 Content Validation: Correctly rejected spoofed PDFs (%PDF missing) and executable extensions (.exe).
 Text/Markdown Support: Accepted valid UTF-8 plain text and markdown documents.
 Checksum Deduplication: Blocked duplicate document uploads in the same organization (409 Conflict).
 Idempotency-Key Handling: Repeated uploads with identical idempotency key returned matching document ID.
 Tenant Isolation: Document status access across organizations rejected with 403 Forbidden.
 Security Masking: Confirmed that internal storage_key is excluded from client JSON envelopes.
 Deletion Scheduling: DELETE /documents/{id} set status to 'deleting' and created deletion outbox event.
 Ingestion Retry: Retrying failed documents reset status to 'pending'; retrying non-failed docs was rejected.

6. Repository Structure in 'Knowledge AI'

The codebase maintains clear boundaries across domain boundaries:

Directory / File	Purpose
cmd/api/main.go	HTTP entrypoint initializing Fiber v2, repositories, handlers, and health probes.
cmd/worker/main.go	Background process initializing Asynq server, Redis client, and graceful shutdown.
internal/documents/models.go	Document, IngestionJob, OutboxEvent, and IdempotencyRecord models.
internal/documents/repository.go	Storage contract for transactional document and outbox operations.
internal/documents/postgres_repo.go	PostgreSQL implementation utilizing pgxpool transactions and outbox events.
internal/documents/memory_repo.go	Thread-safe in-memory repository for unit and integration testing.
internal/documents/service.go	Streaming uploads, SHA-256 calculation, MIME validation, and orphan cleanup.
internal/documents/handler.go	HTTP endpoints: POST upload, GET status, GET list, DELETE, and POST retry.
internal/platform/storage/storage.go	Atomic local filesystem storage driver writing via temporary files.
internal/platform/storage/memory.go	Thread-safe in-memory storage implementation for tests.
tests/unit/documents_test.go	Unit test suite verifying streaming upload, MIME checks, idempotency, and isolation.

7. Instructions for Pushing to GitHub

Phase 3 is complete, tested, and verified against Checkpoint 3. Run the following git commands inside '/home/lucifer/Knowledge AI' to push the updates to your repository:

- **Step 1:**Stage all new and updated files: git add .
- **Step 2:**Commit Phase 3 changes: git commit -m 'feat(phase-3): document management, streaming uploads, and transactional outbox'
- **Step 3:**Push to remote repository: git push origin main